Proteus: Towards Accurate and Low-overhead In-Network Malicious Traffic Detection

Longlong Zhu
Zhejiang University
Hangzhou, Zhejiang, China
kikionepin@gmail.com

Linying Zheng
Zhejiang University
Hangzhou, Zhejiang, China
zhenglinying0106@gmail.com

Qing Shu
Zhejiang University
Hangzhou, Zhejiang, China
singletonshu@gmail.com

Zedi Chen
Zhejiang University
Hangzhou, Zhejiang, China
chenzedi@zju.edu.cn

Jiashuo Yu
Fuzhou University
Fuzhou, Fujian, China
joshuayu.2048@gmail.com

Yuhan Wu
Fuzhou University
Fuzhou, Fujian, China
yharim.io@gmail.com

Shaopeng Zhou
Zhejiang University
Hangzhou, Zhejiang, China
abnerzhou@zju.edu.cn

Hongyan Liu
Fuzhou University
Fuzhou, Fujian, China
hyliu20@zju.edu.cn

Dong Zhang
Fuzhou University
Fuzhou, Fujian, China
zhangdong@fzu.edu.cn

Chunming Wu*
Zhejiang University
Hangzhou, Zhejiang, China
wuchunming@zju.edu.cn

Xiang Chen*
Zhejiang University
Hangzhou, Zhejiang, China
wasdnsxchen@gmail.com

Abstract

Network intrusion detection systems (NIDS) are essential for web security by identifying and dropping malicious traffic. Existing in-network NIDS leverage the Tbps-level packet processing capability of programmable switches to achieve high-speed flow classification. They translate complex trained machine learning models to decision trees (DTs), where DTs are deployed on programmable switches via single-DT or multiple-DT deployment. However, they face a fundamental trade-off: single-DT deployment suffers from low classification accuracy due to over-pruning of trees, while multiple-DT deployment suffers from high overhead due to deploying multiple tree replicas. In this paper, we propose Proteus, an in-network malicious traffic detection system that achieves both high classification accuracy and low overhead. Its key idea is to split the original DT into critical and normal sub-trees, where these sub-trees have different impacts on overall accuracy. More precisely, Proteus first splits a DT into one critical and several normal sub-trees for adapting to the accuracy requirement and switch resource budgets. Second, it minimizes coordination overhead between sub-trees while ensuring full flow coverage via mixed-integer linear programming. Third, it dynamically reallocates or migrates sub-trees to adapt to changing resources by monitoring both classification accuracy and switch resource changes. Testbed experiments with 12.8 Tbps programmable switches show that Proteus improves classification accuracy, reduces switch resource consumption, and reduces classification latency.

CCS Concepts

• **Security and privacy → Network security**.

Keywords

Malicious traffic detection; Programmable switches; Decision tree splitting

ACM Reference Format:
Longlong Zhu, Linying Zheng, Qing Shu, Zedi Chen, Jiashuo Yu, Yuhan Wu, Shaopeng Zhou, Hongyan Liu, Dong Zhang, Chunming Wu, and Xiang Chen. 2026. Proteus: Towards Accurate and Low-overhead In-Network Malicious Traffic Detection. In *Proceedings of the ACM Web Conference 2026 (WWW '26), April 13–17, 2026, Dubai, United Arab Emirates.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3774904.3792381

1 Introduction

Network intrusion detection systems (NIDS) are essential for web security by identifying and dropping malicious traffic [28]. To identify malicious traffic in real-time, prior studies train machine learning (ML) models and deploy ML inference tasks onto the programmable switches that offer Tbps-level packet processing capability, i.e., in-network NIDS [4, 15, 21, 28, 34, 40, 41]. For example, the latency of in-network flow classification is 500× lower than executing that in end-hosts [16].

Among ML models, decision trees (DTs) are easier to deploy onto programmable switches since the nature of feature-based classification within DTs well matches the match-action paradigm of programmable switches [35]. Thus, existing in-network NIDS leverage student-teacher knowledge distillation to translate complex

*Chunming Wu and Xiang Chen are the corresponding authors.

This work is licensed under a Creative Commons Attribution 4.0 International License.

The Web Conference'26, Dubai, United Arab Emirates

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04
https://doi.org/10.1145/3774904.3792381

ML models to deployable DTs [33] or directly build DTs [20, 34]. Then, they convert DTs into P4 programs, which are compiled and installed onto switches. Here, since switch resources are scarce, with multiple programs deployed simultaneously, e.g., sketches [1, 13, 29, 39], FWs [18, 36], and NATs [17, 37], fewer resources are leaved for DTs, leading to deployment failures.

However, existing in-network NIDS solutions that reshape DTs to better fit the limited switch resources fail to achieve high classification accuracy and low overhead simultaneously . We classify the existing solution into single-DT and multiple-DT solutions.

First, the single-DT solutions [15, 33, 35, 41] suffer from classification accuracy drops due to over-pruning. More precisely, they compress DTs to reduce their demand for switch resources: they automatically prune the original DT to a smaller one based on the resource target and then deploy the pruned DT onto a single switch, e.g., Dryad [35] combines hard and soft pruning techniques to compress DTs while maintaining high classification accuracy. However, they suffer from classification accuracy drops due to over-pruning. Here, over-pruning frequently happens under limited switch resource budgets, where nodes exceeding the depth limit are merged into leaf nodes or directly pruned. Since there are only limited available switch ASIC pipeline stages, they need to prune most of DT layers, which drops the classification accuracy.

Second, the multiple-DT solutions suffer from high overheads since they simultaneously deploy multiple tree copies. More precisely, they distribute the input DT across multiple switches: they split the original DT into multiple identical sub-trees and deploy these sub-trees onto multiple switches [5, 28, 40, 42]. In this case, these sub-trees with lower switch resource requirements are easier to deploy and can collectively handle the classification tasks. However, they suffer from high overheads. Here, they typically split a DT into multiple sub-trees and deploy all copies across switches to ensure coverage. Thus, they suffer from high switch resource consumption as multiple tree copies occupy high SRAM, TCAM, and stage resources. Further, they also encounter high coordination overhead between sub-trees, e.g., high latency, as traffic must traverse all sub-trees to complete classification tasks.

In this paper, we propose Proteus, an in-network malicious traffic detection system that achieves both high classification accuracy and low overhead. Its key idea is to split the original DT into one critical sub-tree and several normal sub-trees, where the critical sub-tree contains high-importance classification rules while normal sub-trees handle lower-importance ones. Thanks to the heterogeneous splitting, Proteus preserves high classification accuracy while reducing resource consumption and coordination overhead compared to deploying multiple identical tree copies.

However, we still face three challenges. First, **how to split** the DT to preserve high classification accuracy while minimizing switch resource consumption? Here, identifying which parts of the tree are critical for accuracy and which can be simplified without significant accuracy loss is challenging. Second, **how to deploy** heterogeneous sub-trees onto switches to minimize coordination overhead while ensuring full flow coverage? Here, it is non-trivial to place critical and normal sub-trees across switches to reduce inter-switch communication overhead while guaranteeing that all flows can be properly classified. Third, **how to tune** deployment of sub-trees when switch resources change? Here, it is hard to online tune sub-tree deployment accurately and fast to preserve runtime accuracy.

In response, we design Proteus with three key steps, i..e, decision tree splitting, sub-tree deployment, and online tuning.

Step#1: decision tree splitting (§4.1**).** Proteus splits the input decision tree into heterogeneous sub-trees, i.e., one critical and several normal sub-trees, based on node importance. Here, the critical sub-tree contains high-importance classification rules that significantly impact accuracy, while normal sub-trees handle lower-importance ones. In this case, Proteus preserves high classification accuracy by ensuring that every flow passes through at least one critical sub-tree with any necessary normal sub-trees.

Step#2: sub-tree deployment (§4.2**).** Proteus addresses the deployment problem of heterogeneous sub-trees via a two-stage optimization framework, where Proteus aims to achieve full flow coverage while minimizing the number of deployed critical sub-trees and coordination overhead between sub-trees. More precisely, in the first stage, Proteus formulates and solves a mixed-integer linear programming (MILP) problem to minimize the number of critical sub-tree deployments while ensuring every flow passes through at least one critical sub-tree, thus it can achieve full flow coverage with minimal resource consumption. In the second stage, based on critical sub-tree deployment decisions, Proteus formulates and solves another MILP problem to deploy normal sub-trees, where it minimizes coordination overhead between sub-trees.

Step#3: online tuning (§4.3**).** Proteus handles dynamic changes in switch resource budgets by online tuning. More precisely, Proteus monitors the classification accuracy of sub-trees and switch resource budgets. When suffering from resource conflicts triggered by newly installed programs, Proteus executes two-phase tuning: (1) it reallocates resources of the sub-tree in a conflict-aware manner. For example, in the case of SRAM resource conflicts, it increases the tree depth to trade stage occupation for the SRAM resource. (2) If the switch resource budget is still insufficient after resource reallocation, it splits the critical sub-tree into two critical sub-trees or deactivates normal sub-trees on the conflicted switch. Here, it preserves 100% flow coverage of critical sub-trees while minimizing coordination overhead between critical and normal sub-trees.

We make the following contributions in this paper.

- We propose Proteus, an in-network malicious traffic detection system with high classification accuracy and low overheads (§3).
- We propose an algorithm that splits a DT into one critical sub-tree and several normal sub-trees (§4.1), an optimization framework for optimal sub-tree deployment (§4.2), and an online tuning approach to handle switch resource changes (§4.3).
- We implemented a prototype of Proteus in our testbed. The experimental results indicate that Proteus improves the accuracy from 67.8% to 78.8%, reduces the switch resource consumption by up to 82.8%, and reduces latency by up to 74.5% (§5).

2 Background and Motivation

2.1 In-network NIDS with DTs

In-network NIDS. Web system faces escalating threats from sophisticated attacks that exploit vulnerabilities in web applications and services. NIDS are essential for detecting malicious activities

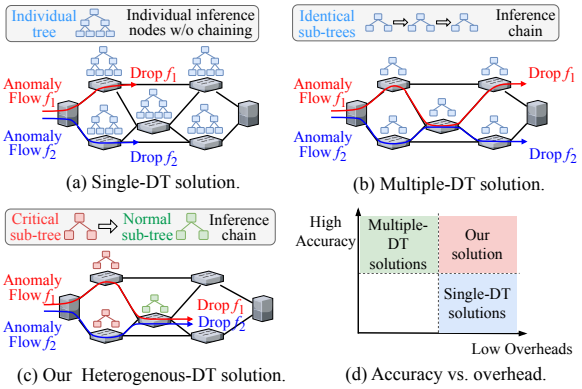


Figure 1: Comparison of Proteus with existing solutions.

and anomalies in the web system [28]. Traditional NIDS are executed on a specific server, where packets are mirrored from switches to analysis servers and then processed in these servers. However, this paradigm suffers from high detection latency since it needs to make detection decisions in the remote servers [16]. With the emergence of programmable switches, researchers offload NIDS directly to programmable switches, based on which they detect network anomalies without packet detours to external devices, i.e., in-network NIDS. Here, thanks to the Tbps-level packet processing capability of programmable switches [2, 8, 17, 30, 37], in-network NIDS achieves microsecond-level detection time [16].

DT-based in-network NIDS. Among machine learning models, decision trees (DTs) have become increasingly popular for in-network NIDS due to their compatibility with programmable switches [32, 34, 38, 41]. More precisely, compared to complex models like graph neural networks [31], the hierarchical structure of feature-based decisions naturally matches the match-action paradigm of programmable switches [15, 34]. The typical workflow for DT-based in-network NIDS involves four steps: (1) It trains a DT using network traffic data with labeled classes. (2) It converts the trained DT into a network program by P4 language [25], where tree nodes are mapped to match-action table entries within the switch. (3) The program is compiled and installed onto the target programmable switch. (4) This program runs on switches to classify incoming packets and detect anomaly packets.

2.2 Limitation of Existing Solutions

We illustrate why existing solutions fail to achieve accurate and low-overhead in-network malicious traffic detection.

The single-DT solutions [4, 15, 33, 35, 41] aim to reduce the resource consumption of the deployed decision tree by selectively pruning tree nodes while minimizing accuracy degradation. The key idea is to identify and prune unimportant nodes or levels within the decision tree. More precisely, they evaluate node importance based on some metrics such as information gain and classification error. Then, they execute pruning-based techniques like cost-complexity pruning or depth-based pruning. Next, they generate a simplified program converted from the pruned tree to reduce switch resource consumption. For example, Dryad [35] executes hard-pruning, i.e., removing nodes beyond a certain depth, and soft-pruning, i.e., selectively merging nodes based on their contribution to accuracy.

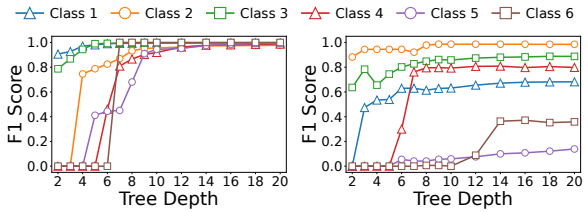


Figure 2: We train a 20-level decision tree for two typical intrusion detection datasets and measure the F1 score of six selected types when tree depth increases.

However, they suffer from classification accuracy drops since they over-prune the decision tree to fit the limited switch resource budget. More precisely, they train a large decision tree or other complex models. Then, they convert the trained model into a simple decision tree that can be deployed onto switches with the match-action paradigm. When suffering from a significant reduction of switch resource budgets, they have to over-prune the decision tree to adapt to changed switch resource budgets. In this case, they suffer from significant classification accuracy drops since critical decision paths for detecting sophisticated malicious traffic are pruned.

The multiple-DT solutions [5, 28, 40, 42] split a large decision tree into multiple smaller sub-trees that can be distributed across multiple switches. The key idea is to decompose the classification task into multiple components that require fewer switch resources. More precisely, they first split the original decision tree into multiple sub-trees. Then, they convert each sub-tree into a separate P4 program and deploy these programs across switches. For example, DINC [40] splits decision trees at specific depth levels and generates multiple identical sub-programs that can be deployed across switches. In this case, each sub-program that handles a subset of classification features is chaining, where the classification results are aggregated via the classification chain for security analysis.

However, they suffer from high overhead since they simultaneously deploy multiple tree copies. More precisely, to maintain classification accuracy when the original decision tree exceeds the resource capacity of a single switch, these solutions typically execute a static decision tree splitting strategy that splits the input decision tree into several identical sub-trees, where they distribute these identical sub-trees across switches to form a classification chain. However, deploying multiple identical replicas leads to high switch resource consumption and coordination overhead between sub-trees. First, these solutions must deploy multiple sub-trees across multiple switches to collectively perform the complete classification logic, where the redundant deployment of trees increases the total switch resource overheads. Second, the classification chain requires packets to traverse multiple switches sequentially, which introduces significant detection latency and inter-switch packet header vector (PHV) overhead.

2.3 Observations and Opportunities

In contrast to existing solutions, we aim to achieve both accurate and low-overhead in-network malicious traffic detection with distributed decision trees. In response, we take a closer look at the classification accuracy of in-network decision trees.

Observation. We obtain the key observation that the marginal benefits of tree nodes on overall classification accuracy diminish.

Settings. We evaluate the performance of decision trees by measuring their classification accuracy.We take two security datasets as input, i.e., CICIDS [26] and UNSW datasets [24], where they contain a wide spectrum of typical web security threats, such as the brute-force attack, Heartbleed, scanning, and DoS attacks.

Experiment results. We change the trained decision tree with different tree depth limitations and measure their classification accuracy. In Figure 2, we observe that as tree depth limitation increases from 2 layers to 20 layers, the contribution of terminal tree layers to overall classification accuracy gradually decreases. More precisely, the first few layers of the tree contain the most important classification features and can handle the majority of traffic with high classification accuracy. While deeper nodes can handle edge cases to achieve fine-grained classification, their marginal benefit to overall accuracy is limited. For example, we find that the top 60% tree layer can achieve >90% maximum classification accuracy, while the remaining 40% of nodes only provide an additional 5-10% improvement in overall classification accuracy. The classification accuracy still drop sharply under over-pruning. For example, in Figure 2(a), leaving only the top 30% of layers decreases classification accuracy, F1 score, from 1.0 to 0.0 for specific classification types.

Takeaway. The non-uniform contribution of tree layers reveals that not all tree nodes are equally critical, which suggests we can achieve near-optimal accuracy without deploying the entire tree.

Opportunities. In light of our observations, we identify the opportunity to achieve accurate and low-overhead in-network malicious traffic detection by splitting trees based on the impact of layer on overall accuracy. More precisely, thanks to the uneven contribution of tree layers to accuracy, we can split the original tree into critical and normal sub-trees according to user accuracy requirements. Critical sub-trees contain high-importance classification rules and must be deployed to guarantee basic accuracy requirements, while normal sub-trees handle low-importance classification rules and can be flexibly deployed based on resource budgets across switches. Thus, we can **preserve high classification accuracy** by routing flow pass through at least one critical and normal sub-trees. Furthermore, compared to deploying multiple identical sub-trees, we can **deploy fewer sub-trees** to reduce switch resource consumption and coordination overheads between sub-trees.

3 Overview of Proteus

We propose Proteus, an in-network malicious traffic detection system that achieves high classification accuracy and low overhead.

Key ideas. Based on our observations in § 2.3, Proteus addresses the limitations of existing solutions through a heterogeneous decision tree splitting approach. To avoid deploying multiple replicas of sub-trees like existing multiple-DT solutions or over-pruning like existing single-DT solutions, Proteus splits the original DT into *one critical sub-tree* and *several normal sub-trees* with different importance levels. Here, the critical sub-tree contains high-importance classification rules that significantly impact overall accuracy and should be deployed to ensure basic accuracy requirements, while normal sub-trees handle lower-importance classification rules and can be flexibly deployed based on available switch resources. By

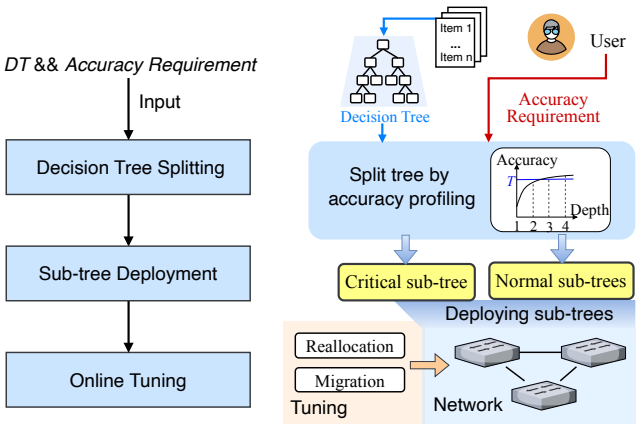


Figure 3: Proteus overview.

this heterogeneous splitting, Proteus preserves high classification accuracy while reducing both switch resource consumption and coordination overhead via deploying fewer sub-trees.

Challenges. We still face three challenges.

C#1: How to split the decision tree to maintain high accuracy while minimizing resource consumption? However, it is non-trivial to split a tree to identify the minimum critical sub-tree that guarantees baseline accuracy while minimizing normal sub-trees to avoid both over-splitting, which suffers from excessive coordination overhead, and under-splitting, which drops accuracy.

C#2: How to deploy the heterogeneous sub-trees onto switches to achieve full flow coverage while minimizing coordination overhead? However, it is non-trivial to balance the switch resource consumption of critical sub-trees, full flow coverage, and inter-sub-tree coordination overhead, where we need to deploy fewer critical sub-trees to guarantee coverage of all flows and then deploy normal sub-trees to improve accuracy while reducing coordination overheads between critical and normal sub-trees.

C#3: How to tune deployment decisions when switch resources change? Since switch resource budgets change, we need to online tune sub-tree deployment to preserve runtime classification accuracy. However, it is non-trivial to maintain classification accuracy after tuning while computing new deployment decisions fast.

Design overview. As is shown in Figure 3, Proteus addresses these challenges through three key steps.

Step#1: Decision tree splitting. To address C#1, Proteus splits the input decision tree into one critical sub-tree and multiple normal sub-trees based on accuracy profiling and contribution analysis. More precisely, Proteus first determines the minimum depth for the critical sub-tree, which contains the top layers starting from the root, by layer-wise profiling on validation data until meeting the user-specified accuracy threshold. Second, it evaluates each branch extending beyond this minimum depth, where Proteus measures both overall accuracy gain and per-class accuracy gain to identify high-contribution branches. Third, it selectively builds normal sub-trees from branches that exceed contribution thresholds to minimize the total number of sub-trees while preserving accuracy.

Step#2: Sub-tree deployment. To address C#2, Proteus addresses the deployment problem of heterogeneous sub-trees via a two-stage optimization framework, where Proteus formulates two MILP problems to achieve minimized switch resource consumption,

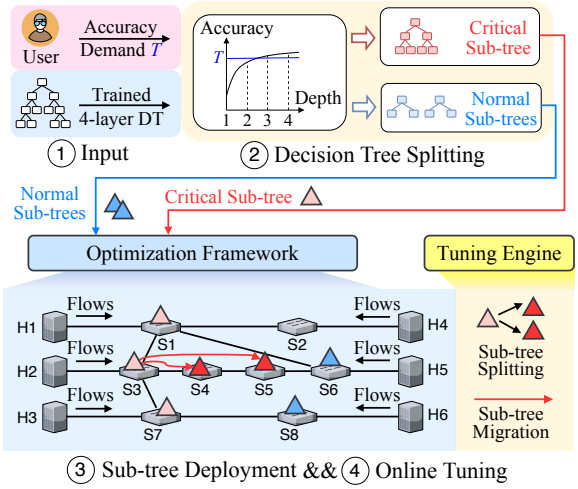


Figure 4: Proteus example.

full flow coverage, and minimized coordination overhead. More precisely, Proteus takes the split sub-trees, network topology, switch resource budgets, and flow routing paths as inputs. In the first optimization stage, Proteus minimizes the number of critical sub-tree deployments while ensuring every flow passes through at least one critical sub-tree. In the second optimization stage, with the obtained critical sub-tree decisions, Proteus optimizes normal sub-tree deployment to minimize coordination overhead between sub-trees. Proteus formulates both optimization stages as MILP problems and computes optimal deployment decisions sequentially.

Step#3: Online tuning. To address C#3, Proteus handles dynamic changes in switch resource budgets. More precisely, Proteus continuously monitors sub-tree classification accuracy and switch resource usage. When resource conflicts are detected, Proteus first executes resource reallocation via the trade-off of resource consumption. For example, for SRAM/TCAM resource conflicts, Proteus trades stage occupation for reduced SRAM/TCAM resource consumption by increasing tree depth. Second, if reallocation is insufficient, Proteus executes different migration strategies based on sub-tree types, i.e., critical sub-trees are split into two smaller critical sub-trees and redeployed, while normal sub-trees are deactivated on conflicted switches and reactivated on other switches. When migrating, Proteus employs a greedy-based heuristic to fast compute near-optimal deployment decisions.

Example. Figure 4 illustrates how Proteus works. Given a trained decision tree with 4 layers, network topology, switch resource budgets, and an 80% accuracy requirement from users.

First, Proteus profiles the decision tree on validation data and determines that a critical sub-tree with layers 1-3 achieves 85% accuracy. It then identifies bridge nodes at layer 3 whose classification decisions need to be encoded into packet header vectors for inter-sub-tree coordination. After that, it generates one critical sub-tree of layers 1-3 and two normal sub-trees for layers 3-4.

Second, Proteus deploys critical sub-tree replicas on resource-abundant switches S1, S3, and S7 to ensure 100% flow coverage, and then deploys normal sub-trees on switches S6 and S8 to utilize residual resources and improve overall classification accuracy. Proteus also minimizes coordination overhead. During online classification, the critical sub-tree encodes its intermediate classification state into packet header vectors (PHV), based on which packets are routed to the corresponding normal sub-trees for fine-grained classification.

Third, when new sketches consuming 16 MB TCAM resources are installed on S3 with 8 MB TCAM resource budgets, Proteus first attempts to reallocate resources by increasing tree depth from 3 to 5 layers. Here, it reduces TCAM resource consumption while occupying 2 additional switch pipeline stages. However, the resource conflicts persist. In response, it splits the critical sub-tree into two smaller critical sub-trees and deploys them on S4 and S5 to maintain full flow coverage while minimizing coordination overhead.

Table 1: Notation of main symbols.

Symbol	Description
T	Input decision tree
T^c	Critical sub-tree with depth d_{min}
T_j^n	Normal sub-tree j ($j = 1, \ldots, k$)
B	Bridge nodes (i.e., leaf nodes of T^c)
α	User-specified accuracy threshold
θ	Threshold for overall accuracy gain
θ'	Threshold for per-class accuracy gain
V_{test}	Validation dataset for accuracy profiling
$\Delta acc_{overall}$	Overall accuracy gain of a branch
$\Delta acc_{per-class}$	Per-class accuracy gain of a branch
$G(V, E)$	Network topology with switches V and links E
F	Set of flows
$path(f)$	Routing path of flow f
b_f	Bandwidth requirement of flow f
C_e	Bandwidth capacity of link e
R_i	Resource budget of switch i
r^c	Resource requirement of critical sub-tree T^c
r_j^n	Resource requirement of normal sub-tree T_j^n
x_i^c	Variable indicating if T^c deployed on switch i
x_i^j	Variable indicating if T_j^n deployed on switch i
$dep(B)$	Dependency relations derived from bridge nodes B
$w_{j,l}$	Coordination intensity between sub-trees j and l
$d_{j,l}$	Distance between sub-trees j and l
$y_i^{j,l}$	Variable indicating if sub-trees j and l on switch i

4 Proteus Design

4.1 Decision Tree Splitting

Proteus adapts to different switch resource budgets by splitting a decision tree into one critical sub-tree and multiple normal sub-trees with varying resource demands and accuracy contributions.

Challenge. The key challenge is to maintain high classification accuracy while minimizing the number of normal sub-trees. More precisely, too many normal sub-trees increase deployment complexity and coordination overhead, while too few sub-trees cannot achieve the accuracy requirement.

Solution overview. We design a splitting algorithm that first identifies the minimum depth for the critical sub-tree through layer-wise profiling. Then, it selectively constructs normal sub-trees by evaluating each branch's contribution to both overall and per-class accuracy, since different branches contribute unequally to overall accuracy and per-class accuracy.

We summarize the splitting workflow in Algorithm 1. The details of Algorithm 1 are as follows.

Critical sub-tree identification. Algorithm 1 determines the minimum depth d_{min} where accuracy meets the user-specified

threshold α (Lines 1-7). More precisely, it extracts progressively deeper sub-trees and evaluates their accuracy on validation data V_{test} in a top-down manner. Once accuracy meets α, the search stops, where it generates a critical sub-tree T^c that guarantees baseline accuracy with minimal resources.

Branch-level profiling. After extracting T^c, Algorithm 1 identifies bridge nodes B of T^c (Line 8). Each bridge node may have multiple children, forming distinct branches that extend beyond d_{min} (Line 9). Proteus then evaluates each branch's accuracy contribution on V_{test} (Lines 12-14). Here, it measures both overall accuracy gain $\Delta acc_{overall}$ and per-class accuracy gains $\Delta acc_{per-class}$ for each class in the classification task. For example, in a 6-class flow classification task, a branch may primarily improve accuracy for one specific class while having a negligible impact on others.

Selective normal sub-tree construction. Proteus selectively retains branches as normal sub-trees based on their accuracy contributions (Lines 15-16). Here, a branch becomes a normal sub-tree T_i^n when satisfying (1) its overall accuracy gain exceeds threshold θ, or (2) its accuracy gain for any specific class exceeds threshold θ'. In this way, by pruning low-contribution branches, Proteus minimizes the number of normal sub-trees while preserving accuracy.

Complexity. The time complexity is $O(|B| \cdot C \cdot |V_{test}|)$, where C denote the number of classes. We leave the analysis in the appendix.

4.2 Two-stage Optimization Framework

We design an optimization framework to obtain the optimal deployment of these sub-trees across switches. Our framework comprises two optimization stages. In the first stage, we aim to minimize the number of deployed critical sub-trees to ensure full flow coverage without concentrating traffic on specific switches. In the second stage, we aim to minimize inter-sub-tree coordination overhead. Table 1 summarizes the main symbols in our paper.

Inputs & outputs. Proteus takes the following inputs: (1) the split sub-trees $T^c, T_1^n, \ldots, T_k^n$ and bridge nodes B from Algorithm 1, (2) network topology $G(V, E)$, (3) switch resource budgets R_i for each switch $i \in V$, and (4) flow set F with their routing paths. It outputs deployment decisions $x_i^c \in \{0, 1\}$ that indicate whether the critical sub-tree T^c is deployed on switch i, and $x_i^j \in \{0, 1\}$ that indicate whether the normal sub-tree T_j^n is deployed on switch i.

Optimization stage#1. In the first stage, we formulate the critical sub-tree deployment problem as an MILP problem.

Objective. We aim to minimize the number of deployed critical sub-trees while ensuring full flow coverage, i.e.,

$$\min \quad \sum_{i \in V} x_i^c \tag{1}$$

Constraints. We identify several constraints when deploying critical sub-trees.

(1) Resource. Each switch does not exceed its resources when deploying the critical sub-tree, i.e.,

$$r^c \cdot x_i^c \le R_i, \quad \forall i \in V \tag{2}$$

(2) Link capacity. Flow routing must respect the bandwidth capacity of each network link, i.e.,

$$\sum_{f \in F : e \in path(f)} b_f \le C_e, \quad \forall e \in E \tag{3}$$

Algorithm 1 Decision Tree Splitting

Require: Tree T, accuracy threshold α, validation set V_{test}
Ensure: Critical sub-tree T^c, normal sub-trees $\{T_1^n, \ldots, T_k^n\}$, bridge nodes B

1: $best_{depth} \leftarrow 0$
2: **for** $depth = 1$ to MaxDepth(T) **do**
3: $\quad T_{depth} \leftarrow$ extract sub-tree from root to $depth$
4: $\quad acc \leftarrow$ Evaluate(T_{depth}, V_{test})
5: $\quad$ **if** $acc \ge \alpha$ **then**
6: $\quad\quad best_{depth} \leftarrow depth$, **break**
7: $T^c \leftarrow$ extract sub-tree from root to $best_{depth}$
8: $B \leftarrow$ leaf nodes of T^c
9: $branches \leftarrow$ all sub-trees rooted at children of B
10: Sort $branches$ by accuracy contribution in descending order
11: $T^n \leftarrow \emptyset$
12: **for** each $branch$ in $branches$ **do**
13: $\quad \Delta acc_{overall} \leftarrow$ accuracy gain on V_{test}
14: $\quad \Delta acc_{per-class} \leftarrow$ per-class accuracy gains on V_{test}
15: $\quad$ **if** $\Delta acc_{overall} > \theta$ **or** any $\Delta acc_{per-class} > \theta'$ **then**
16: $\quad\quad T^n \leftarrow T^n \cup \{branch\}$
17: **return** T^c, T^n

(3) Coverage. Every flow should pass through the critical sub-tree to maintain minimum classification accuracy demand, i.e.,

$$\sum_{i \in path(f)} x_i^c \ge 1, \quad \forall f \in F \tag{4}$$

Optimization stage#2. In the second stage, we formulate the normal sub-tree deployment problem as an MILP problem.

Objective. With critical sub-tree deployment decisions from optimization stage#1, we minimize total coordination overhead between sub-trees by optimizing normal sub-tree deployment, i.e.,

$$\min \quad \sum_{i \in V} \sum_{j,l \in c,1,\ldots,k æ \neq l} w_{j,l} \cdot y^{j,l} i \cdot dj, l \tag{5}$$

Constraints. We identify constraints when deploying normal sub-trees. Here, the constraint of link capacity is the same as stage#1.

(1) Resource. Each switch does not exceed its resources when deploying both critical and normal sub-trees, i.e.,

$$r^c \cdot x_i^c + \sum_{j=1}^{k} r_j^n \cdot x_i^j \le R_i, \quad \forall i \in V \tag{6}$$

(2) Semantic dependency. Sub-tree dependencies must be respected along flow paths, i.e.,

$$\sum_{i \in path(f)} x_i^l \ge \sum_{i \in path(f)} x_i^j, \quad \forall f \in F, \forall (j \to l) \in dep(B) \tag{7}$$

Solving. Following established practices in network optimization [7, 10, 22, 40], we employ Gurobi [14] as the standard MILP solver to compute optimal deployment decisions in our framework.

4.3 Online Tuning

Proteus handles dynamic changes in switch resource budgets by continuously monitoring sub-tree performance and executing adaptive resource reallocation or migration for sub-trees.

Challenge. However, it is non-trivial to achieve accurate and timely adaptation to dynamic resource changes due to the trade-off between tuning speed and quality. When resource conflicts are triggered by newly installed programs, fast tuning is essential to maintain runtime classification, but it drops accuracy. Naive approaches like directly removing a sub-tree provide fast conflict resolution but lead to coverage gaps and significant accuracy drops. More accurate

Algorithm 2 Conflict-aware Resource Reallocation

Require: Sub-tree T, current resources $R_{current}$, required resources $R_{required}$, switch s, network state N, bridge nodes B
Ensure: Restructured sub-tree T or migration flag
1: $conflict \leftarrow$ DetectConflict($R_{current}, R_{required}$)
2: **if** $conflict$ = NONE **then**
3: **return** T
4: **while** $conflict$! = NONE **do**
5: **if** $conflict$ = MEMORY **then**
6: $T \leftarrow$ ReduceMemory(T) ▷ Sacrifice stages
7: $R_{required} \leftarrow$ UpdateResources(T)
8: **else if** $conflict$ = STAGE **then**
9: $T \leftarrow$ ReduceStage(T) ▷ Merge nodes bottom-up
10: $R_{required} \leftarrow$ UpdateResources(T)
11: **if** ReallocationFail(T) **then**
12: **return** Migration(T,s,N,B)
13: $conflict \leftarrow$ DetectConflict($R_{current}, R_{required}$)
14: **return** T

solutions like sub-tree migration or fine-grained pruning require long computation time to obtain optimal tuning strategies, during which the classification task operates with suboptimal resource utilization and degraded performance.

Solution overview. We design a two-level framework in the online tuning phase, i.e., the *conflict-aware resource reallocation* for trading off different types of switch resources, and the *migration algorithm* for maintaining 100% flow coverage and preserving classification accuracy. Here, Proteus migrates sub-trees when the local resource reallocation is insufficient. The details are as follows.

Conflict-aware resource reallocation. We first attempt to resolve resource conflicts through local resource optimization without altering the sub-tree deployment decisions.

Insight. We exploit the switch resource trade-offs of in-network decision trees. Specifically, decision trees exhibit configurable stage and memory trade-offs, where we can configure the trade-off by introducing more features or other engineering techniques.

Algorithm. We develop the conflict-aware reallocation algorithm that dynamically reconfigures sub-tree structures to resolve resource conflicts within a single switch (Algorithm 2). More precisely, we first identify the type of resource conflicts by comparing current resource consumption against available capacity (Lines 1-3). For memory resource conflicts (e.g., SRAM and TCAM resources), we trade the occupation of the switch ASIC pipeline stages for more memory resources (Lines 5-7). For stage conflicts, we reduce tree depth by merging nodes down to up. Thus, we can trade memory resources for more idle stages (Lines 8-10). We iteratively execute resource reallocation until either conflict is resolved or choose to execute sub-tree migration when further reallocation becomes infeasible due to switch resource constraints (Lines 11-14).

Sub-tree migration. When resource reallocation cannot resolve resource conflicts, Proteus employs sub-tree migration to relocate conflicted sub-trees to alternative switches.

Insight. Different sub-tree types require different migration strategies. More precisely, critical sub-trees should maintain 100% flow coverage after migration to preserve high accuracy, while normal sub-trees can be deactivated and reactivated in other switches.

Algorithm. We present a sub-tree migration algorithm (Algorithm 3). For critical sub-tree migration, we split it into several smaller sub-trees while maintaining classification semantics to preserve high classification accuracy (Lines 1-8). Then, these sub-trees are deployed onto switches. For normal sub-tree migration, we

Algorithm 3 Sub-tree Migration

Require: Conflicted sub-tree T, switch s, network state N, bridge nodes B
Ensure: Migration decisions MD
1: **if** T is critical sub-tree **then**
2: $T_1, T_2, \ldots, T_k \leftarrow$ SemanticSplit(T, B)
3: $candidates \leftarrow$ EvaluateBenefit($N.switches, T_1, \ldots, T_k$)
4: **for** each $(s_1, s_2, \ldots, s_k) \in candidates$ **do**
5: **if** $\bigwedge_{i=1}^{k}$ CanDeploy(s_i, T_i) **then**
6: **if** FullCover(s_i, T_i) **then**
7: $MD \leftarrow$ DeployMultiple((s_i, T_i))
8: **return** MD
9: **else**
10: Deactivate(s, T)
11: $scores \leftarrow$ CalculateBenefit($N.switches, T$)
12: $candidates \leftarrow$ SortByBenefit($scores$)
13: **for** each $(s_1, s_2, \ldots, s_k) \in candidates$ **do**
14: **if** CanDeploy(s_i, T) **and** $scores[s_i] > \theta$ **then**
15: $MD \leftarrow$ Deploy(s_i, T)
16: **return** MD

deactivate them in the switch that suffers from resource conflict (Line 10). Then, we reactivate them in other switches with enough resource budgets (Lines 11-16). During migration, we aim to minimize the coordination overhead between sub-trees. In this case, we leverage a greedy-based heuristic that selects migration targets to minimize the post-migration coordination cost between sub-trees.

5 Evaluation

We implement a prototype of Proteus in our testbed and evaluate it compared to existing solutions.Our experimental result indicates that Proteus reduces switch resource consumption by up to 82.8%, decreases the coordination overheads between sub-trees by up to 93.6%, improves classification accuracy from 67.8% to 78.8%, and reduce latency by up to 74.5%. We conduct more experiments and leave the results in the appendix.

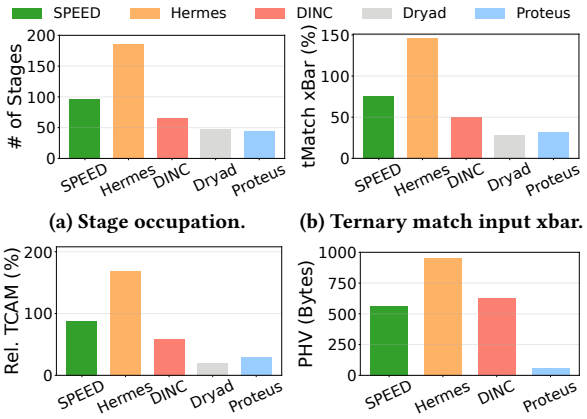
5.1 Experimental Setup

Testbed. We implement a prototype of Proteus our testbed comprising two 12.8 Tbps Intel Tofino switches [2] and two servers. We also simulate large-scale topologies via Mininet [23]. Each switch is configured with random and heterogeneous resource budgets in our large-scale simulation.

Datasets. We conduct experiments on three representative malicious traffic detection datasets, i.e., UNSW dataset [24] for hybrid malicious traffic classification, CICIDS datasets [26], and CICDDoS dataset [27] for intrusion detection. Here, these workloads cover a wide spectrum of typical web security threats, such as the brute-force attack, Heartbleed, scanning, DoS, and DDoS attacks.

Topology. We select two types of real-world topologies that support web services: (1) Wide area network (WAN) topologies from the Internet Topology Zoo [19], which resemble the backbone networks used to deliver web content across geographically distributed locations. We configure each link with 10 Gbps bandwidth [11]. (2) Data center (DC) topologies, i.e., the Fat-Tree topology, which represent modern data centers that host large-scale web applications. We randomly set the link latency between 1 us and 10 us [11].

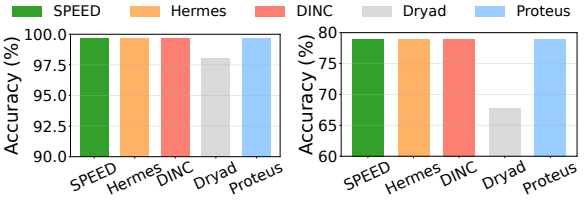
Baselines. We compare Proteus with three types of representative solutions. (1) *Single-DT solutions.* We choose Dryad [35] and fix the tree depth limitation to 6 layers. (2) *Multiple-DT solutions.* We choose DINC [40] that distributes identical-sized sub-trees across switches. (3) *General programs deployment frameworks.* We modify



(a) Stage occupation. (b) Ternary match input xbar.

(c) TCAM resource overhead. (d) Per-packet byte overhead.

Figure 5: Switch resource consumption and inter-sub-tree coordination consumption under Internet2 topology.



(a) CICIDS. (b) UNSW.

Figure 6: Accuracy under different datasets. We leave the full results in our appendix due to page limitations.

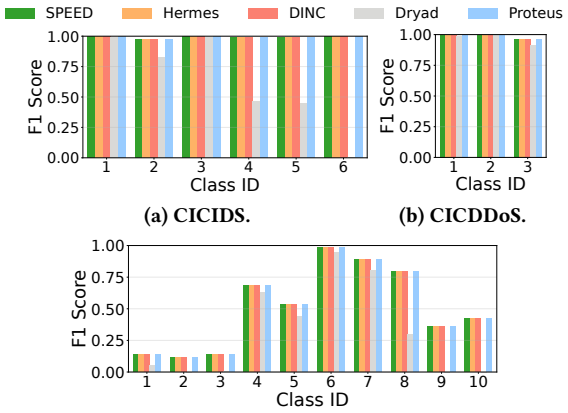
SPEED [9] and Hermes [10] for multiple sub-trees deployment. We leave more details of baselines in the appendix.

5.2 Experimental Results

Switch resource consumption. Figure 5(a)-(c) shows the switch resource consumption for deploying decision trees trained via sklearn [3]. We measure three critical switch resources consumption, including switch ASIC pipeline stages, ternary match input xbar, and TCAM resource consumption. Compared to the existing multiple-DT solution, Proteus reduces stage occupation by up to 76.3%, ternary match input xbar usage by up to 77.8%, and TCAM resource consumption by up to 82.8%. This is because our heterogeneous tree splitting algorithm generates fewer sub-trees by splitting the tree based on both resources and accuracy requirements.

Coordination overhead between sub-trees. To quantify the overhead of sub-tree coordination, we measure the consumption of PHV resources. Figure 5(d) presents the PHV resource consumption of Proteus and comparison solutions. We can see that Proteus reduces PHV resource consumption by up to 93.6%. The reasons are two-fold. First, we build fewer sub-trees, where we require fewer inter-sub-tree contexts to be passed between switches. Second, our optimization framework minimizes the coordination overheads while maintaining full flow coverage. Further, the single-DT solution incurs zero PHV overhead while suffering from accuracy drops due to over-pruning in switches with limited resources.

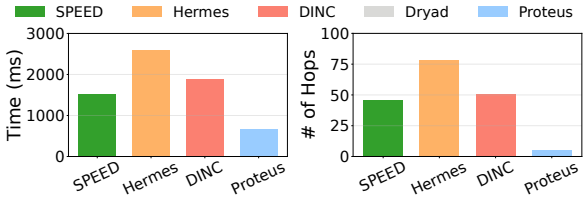
Detection performance. We evaluate detection performance using decision trees trained from three datasets, i.e., CICIDS [26], CICDDoS [27], and UNSW datasets [24]. Figure 6 and Figure 7 show the overall accuracy and per-class F1 scores for these multi-class classification tasks. Compared to the single-DT solution, Proteus



(a) CICIDS. (b) CICDDoS.

(c) UNSW.

Figure 7: F1 score under different datasets.



(a) End-to-end latency. (b) Total inter-sub-tree hops.

Figure 8: Detection speed under real-world WAN topology.

improves overall accuracy from 67.76% to 78.83% on the UNSW dataset (Figure 6). Figure 7 presents per-class F1 scores across three datasets. Here, Proteus maintains high F1 scores across most classes. Thanks to our accuracy-aware splitting algorithm, Proteus also improves F1 scores from 0.0 to 1.0 for minority classes that were completely missed by the single-DT solutions.

Detection speed. We evaluate detection speed in the Internet2 topology. We measure the total number of inter-sub-tree hops and the end-to-end latency to quantify detection speed. Figure 8 shows that Proteus reduces the total hop count by up to 93.6% and end-to-end latency by up to 74.5%. We also evaluate detection speed in a real-world DC topology and leave the results in our appendix.

6 Conclusion

We propose Proteus, an in-network malicious traffic detection system that achieves both high classification accuracy and low overhead. It splits a decision tree into one critical sub-tree and several normal sub-trees, preserving accuracy while reducing the number of normal sub-trees. Then, it computes the optimal deployment decisions to minimize inter-sub-tree coordination overheads while ensuring full flow coverage. Finally, it tunes the deployed sub-trees and deployment decisions to maintain high accuracy and low overhead. Experimental results show that Proteus improves classification accuracy from 67.8% to 78.8%, reduces resource consumption by up to 82.8%, and reduces classification latency by up to 74.5%.

Acknowledgments

This work is supported by the National Key Research and Development (R&D) Program of China (2023YFB2904000) and the National Natural Science Foundation of China (No. U25B2024).

References

[1] Anup Agarwal, Zaoxing Liu, and Srinivasan Seshan. 2022. HeteroSketch: Coordinating Network-wide Monitoring in Heterogeneous and Dynamic Networks. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. USENIX Association, 719–741.

[2] Barefoot Network. Barefoot Tofino. 2025. https://www.barefootnetworks.com/technology/#tofino.

[3] Lars Buitinck, Gilles Louppe, Mathieu Blondel, Fabian Pedregosa, Andreas Mueller, Olivier Grisel, Vlad Niculae, Peter Prettenhofer, Alexandre Gramfort, Jaques Grobler, Robert Layton, Jake VanderPlas, Arnaud Joly, Brian Holt, and Gaël Varoquaux. 2013. API design for machine learning software: experiences from the scikit-learn project. In *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*. 108–122.

[4] Coralie Busse-Grawitz, Roland Meier, Alexander Dietmüller, Tobias Bühler, and Laurent Vanbever. 2022. pForest: In-Network Inference with Random Forests. arXiv:1909.05680 [cs.NI] https://arxiv.org/abs/1909.05680

[5] Beyza Bütün, David De Andres Hernandez, Michele Gucciardo, and Marco Fiore. 2025. DUNE: Distributed Inference in the User Plane. In *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*. 1–10.

[6] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, 785–794.

[7] Xiang Chen, Qun Huang, Peiqiao Wang, Hongyan Liu, Yuxin Chen, Dong Zhang, Haifeng Zhou, and Chunming Wu. 2021. MTP: Avoiding control plane overload with measurement task placement. In *IEEE INFOCOM*. 1–10.

[8] Xiang Chen, Qun Huang, Peiqiao Wang, Zili Meng, Hongyan Liu, Yuxin Chen, Dong Zhang, Haifeng Zhou, Boyang Zhou, and Chunming Wu. 2021. Lightnf: Simplifying network function offloading in programmable networks. In *2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQOS)*. IEEE, 1–10.

[9] Xiang Chen, Hongyan Liu, Qun Huang, Peiqiao Wang, Dong Zhang, Haifeng Zhou, and Chunming Wu. 2020. SPEED: Resource-Efficient and High-Performance Deployment for Data Plane Programs. In *IEEE ICNP*. 1–12.

[10] Xiang Chen, Hongyan Liu, Qingjiang Xiao, Qun Huang, Dong Zhang, Haifeng Zhou, Boyang Zhou, Chunming Wu, Xuan Liu, and Qiang Yang. 2024. Hermes: Low-Overhead Inter-Switch Coordination in Network-Wide Data Plane Program Deployment. *IEEE/ACM Transactions on Networking* (2024), 2842–2857.

[11] Xiang Chen, Qingjiang Xiao, Hongyan Liu, Qun Huang, Dong Zhang, Xuan Liu, Longbing Hu, Haifeng Zhou, Chunming Wu, and Kui Ren. 2024. Eagle: Toward Scalable and Near-Optimal Network-Wide Sketch Deployment in Network Measurement. In *ACM SIGCOMM*. 291–310.

[12] Jerome H. Friedman. 2001. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics* 29, 5 (2001), 1189 – 1232. doi:10.1214/aos/1013203451

[13] Liyuan Gu, Ye Tian, Wei Chen, Zhongxiang Wei, Cenman Wang, and Xinming Zhang. 2023. Per-Flow Network Measurement With Distributed Sketch. *IEEE/ACM Transactions on Networking* (2023).

[14] Gurobi Optimizer. 2025. http://www.gurobi.com.

[15] Syed Usman Jafri, Sanjay Rao, Vishal Shrivastav, and Mohit Tawarmalani. 2024. Leo: Online ML-based Traffic Classification at Multi-Terabit Line Rate. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. USENIX Association, 1573–1591.

[16] Syed Usman Jafri, Sanjay Rao, Vishal Shrivastav, and Mohit Tawarmalani. 2024. Leo: online ML-based traffic classification at multi-terabit line rate. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

[17] Xin Jin, Xiaozhou Li, Haoyu Zhang, Robert Soulé, Jeongkeun Lee, Nate Foster, Changhoon Kim, and Ion Stoica. 2017. Netcache: Balancing key-value stores with fast in-network caching. In *ACM SOSP*. 121–136.

[18] Changhun Jung, Sian Kim, Rhongho Jang, David Mohaisen, and DaeHun Nyang. 2022. A Scalable and Dynamic ACL System for In-Network Defense. In *ACM SIGSAC Conference on Computer and Communications Security*. 1679–1693.

[19] Simon Knight, Hung X Nguyen, Nickolas Falkner, Rhys Bowden, and Matthew Roughan. 2011. The internet topology zoo. *IEEE Journal on Selected Areas in Communications* 29, 9 (2011), 1765–1775.

[20] Ruoyu Li, Qing Li, Yu Zhang, Dan Zhao, Xi Xiao, and Yong Jiang. 2024. Genos: General In-Network Unsupervised Intrusion Detection by Rule Extraction. In *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*.

[21] Jiaye Lin, Qing Li, Guorui Xie, Yong Jiang, Zhenhui Yuan, Changlin Jiang, and Yuan Yang. 2023. In-Forest: Distributed In-Network Classification with Ensemble Models. In *2023 IEEE 31st International Conference on Network Protocols (ICNP)*. IEEE, 1–12.

[22] Hongyan Liu, Xiang Chen, Qun Huang, Guoqiang Sun, Peiqiao Wang, Dong Zhang, Chunming Wu, Xuan Liu, and Qiang Yang. 2024. Toward Resource-Efficient and High- Performance Program Deployment in Programmable Networks. *IEEE/ACM Transactions on Networking* (2024), 4270–4285.

[23] Mininet. 2025. http://mininet.org/.

[24] Nour Moustafa and Jill Slay. 2015. UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In *2015 Military Communications and Information Systems Conference (MilCIS)*. 1–6.

[25] P4C. [n. d.]. https://github.com/p4lang/p4c.

[26] Iman Sharafaldin, Arash Habibi Lashkari, Ali A Ghorbani, et al. 2018. Toward generating a new intrusion detection dataset and intrusion traffic characterization. *ICISSp* 1, 2018 (2018), 108–116.

[27] Iman Sharafaldin, Arash Habibi Lashkari, Saqib Hakak, and Ali A. Ghorbani. 2019. Developing Realistic Distributed Denial of Service (DDoS) Attack Dataset and Taxonomy. In *2019 International Carnahan Conference on Security Technology (ICCST)*. 1–8.

[28] Zhenning Shi, Dan Zhao, Yijia Zhu, Guorui Xie, Qing Li, and Yong Jiang. 2025. Helios: Learning and Adaptation of Matching Rules for Continual In-Network Malicious Traffic Detection. In *Proceedings of the ACM on Web Conference 2025*. 2319–2329.

[29] Haifeng Sun, Qun Huang, Jinbo Sun, Wei Wang, Jiaheng Li, Fuliang Li, Yungang Bao, Xin Yao, and Gong Zhang. 2024. {AutoSketch}: Automatic {Sketch-Oriented} Compiler for Query-driven Network Telemetry. 1551–1572. https://www.usenix.org/conference/nsdi24/presentation/sun

[30] Tofino2. [n. d.]. https://www.intel.com/content/www/us/en/products/network-io/programmable-ethernet-switch/tofino-2-series.html.

[31] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. 2021. A Comprehensive Survey on Graph Neural Networks. *IEEE Transactions on Neural Networks and Learning Systems* 32, 1 (Jan. 2021), 4–24. doi:10.1109/tnnls.2020.2978386

[32] Guorui Xie, Qing Li, Chupeng Cui, Ruoyu Li, Lianbo Ma, Zhuyun Qi, and Yong Jiang. 2024. Intelligent In-Network Attack Detection on Programmable Switches with Soterv2. *IEEE Transactions on Dependable and Secure Computing* (2024), 1–17.

[33] Guorui Xie, Qing Li, Yutao Dong, Guanglin Duan, Yong Jiang, and Jingpu Duan. 2022. Mousika: Enable General In-Network Intelligence in Programmable Switches by Knowledge Distillation. In *IEEE INFOCOM 2022 - IEEE Conference on Computer Communications*.

[34] Guorui Xie, Qing Li, Jiaye Lin, Gianni Antichi, Dan Zhao, Zhenhui Yuan, Ruoyu Li, and Yong Jiang. 2023. Dryad: Deploying Adaptive Trees on Programmable Switches for Networking Classification. In *2023 IEEE 31st International Conference on Network Protocols (ICNP)*. 1–11.

[35] Guorui Xie, Qing Li, Jiaye Lin, Gianni Antichi, Dan Zhao, Zhenhui Yuan, Ruoyu Li, and Yong Jiang. 2023. Dryad: Deploying Adaptive Trees on Programmable Switches for Networking Classification. In *IEEE International Conference on Network Protocols (ICNP)*.

[36] Wafik Zahwa, Abdelkader Lahmadi, Michael Rusinowitch, and Mondher Ayadi. 2024. In-Network ACL Rules Placement using Deep Reinforcement Learning. In *2024 IEEE International Mediterranean Conference on Communications and Networking (MeditCom)*. 341–346. doi:10.1109/MeditCom61057.2024.10621188

[37] Lior Zeno, Ang Chen, and Mark Silberstein. 2024. In-Network Address Caching for Virtual Networks. In *ACM SIGCOMM (ACM SIGCOMM '24)*. 735–749.

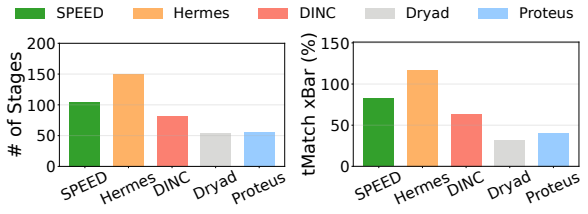
[38] Mai Zhang, Lin Cui, Xiaoquan Zhang, Fung Po Tso, Zhen Zhang, Yuhui Deng, and Zhetao Li. 2025. Quark: Implementing Convolutional Neural Networks Entirely on Programmable Data Plane.

[39] Yinda Zhang, Peiqing Chen, and Zaoxing Liu. 2024. OctoSketch: Enabling Real-Time, Continuous Network Monitoring over Multiple Cores. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 1621–1639.

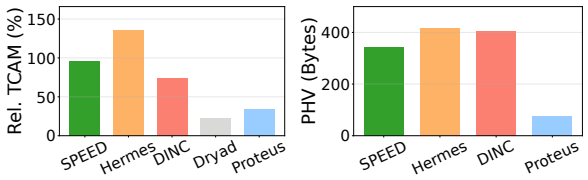
[40] Changgang Zheng, Haoyue Tang, Mingyuan Zang, Xinpeng Hong, Aosong Feng, Leandros Tassiulas, and Noa Zilberman. 2023. DINC: Toward Distributed In-Network Computing. *ACM CONEXT* (2023).

[41] Changgang Zheng, Zhaoqi Xiong, Thanh T. Bui, Siim Kaupmees, Riyad Bensoussane, Antoine Bernabeu, Shay Vargaftik, Yaniv Ben-Itzhak, and Noa Zilberman. 2024. IIsy: Hybrid In-Network Classification Using Programmable Switches. *IEEE/ACM Trans. Netw.* 32, 3 (2024), 2555–2570.

[42] Guangmeng Zhou, Zhuotao Liu, Chuanpu Fu, Qi Li, and Ke Xu. 2023. An Efficient Design of Intelligent Network Data Plane. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, 6203–6220.



(a) Stage occupation. (b) Ternary match input xbar.



(c) TCAM resource overhead. (d) Per-packet byte overhead.

Figure 9: Switch resource consumption and inter-sub-tree coordination consumption under Fat-Tree topology.

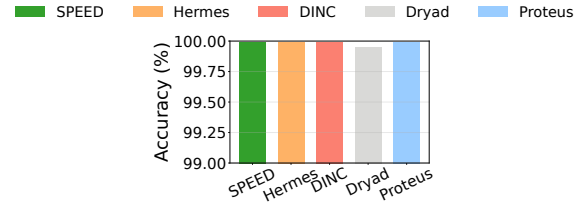
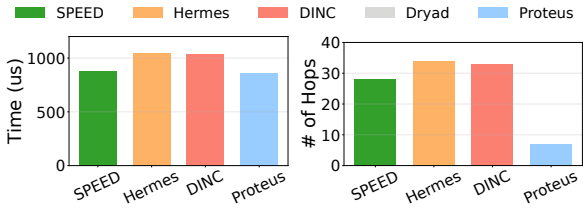


Figure 10: Accuracy under CICDDoS datasets.



(a) End-to-end latency. (b) Total inter-sub-tree hops.

Figure 11: Detection speed under real-world DC topology, i.e., a Fat-Tree topology.

7 Appendix

7.1 More Details of Baselines

Baselines. We compare Proteus with three types of representative solutions. (1) *Single-DT solutions.* We choose Dryad [35] that prunes trees to fit individual switch constraints. Here, we fix the tree depth limitation to 6 layers and prune trained decision trees via hard pruning of Dryad. To detect malicious traffic, we deploy the pruned tree onto multiple switches, where we set the number of switches used to 20% total switches. (2) *Multiple-DT solutions.* We choose DINC [40] that distributes identical-sized sub-trees across switches. Here, we split a decision tree into multiple identical-sized sub-trees, where we execute splitting manually when the resource requirement of sub-trees exceeds the switch capacity. (3) *General programs deployment frameworks.* We modify SPEED [9] and Hermes [10] for multiple sub-trees deployment. Here, SPEED aims to minimize stage occupation and latency. Hermes aims to minimize inter-switch per-packet byte overhead by heuristics.

7.2 More Experimental Rusults

Switch resource overheads. We conduct more experiments on the overheads of Proteus and comparison solutions. We measure the switch resource overheads and inter-sub-tree coordination overheads under a real-world data center topology, i.e., a Fat-Tree topology. Figure 9 shows that Proteus achieves low overhead. It reduces the stage occupation by up to 62.7%, ternary match input xbar usage by up to 65.3%, and TCAM resource consumption by up to 74.5%. It also reduces the PHV resource overhead by up to 79.4%.

Accuracy. We supplement the result of the accuracy of Proteus against the comparison solution under the CICDDoS traces [27]. Figure 10 shows that Proteus maintains high classification accuracy across traces.

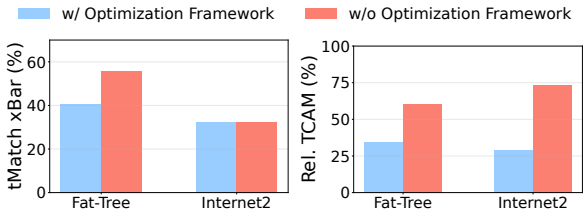
Detection speed under more topologies. We conduct more experiments to evaluate the detection speed under more real-world topologies. We evaluate detection speed in the Fat-Tree topology, a real-world DC topology. We measure the total number of inter-sub-tree hops and the end-to-end latency to quantify detection speed. Figure 11 shows that Proteus reduces the total hop count by up to 79.4% and end-to-end latency by up to 18.4%.

7.3 Ablation Studies

We evaluate the contribution of the two-stage optimization framework in Proteus. More precisely, we compare Proteus's two-stage optimization framework with a greedy-based heuristic, i.e., Proteus w/o optimization framework. It optimizes deployment by greedily deploying sub-trees for each flow. For critical sub-trees, it adopts a first-fit strategy along the flow path, where it either reuses an existing critical sub-tree or places a new one at the first resource-sufficient switch. For normal sub-trees, it employs a nearest-fit strategy, where it selects the nearest switches with sufficient resources. Figure 12 shows that compared to the heuristic, the optimization framework in Proteus reduces ternary match input xbar by up to 48.9% and reduces TCAM resource overhead by up to 60.2%.

7.4 Algorithm Complexity Analysis

We analyze the time complexity of Algorithm 1. Let d denote the maximum depth of tree T, $|V_{test}|$ denote the validation set size, $|B|$ denote the number of branches, h_{avg} denote the average branch height, and C denote the number of classes. Algorithm 1 comprising four phases: (1) *Critical sub-tree extraction* (Lines 2-7) iterates through at most d depth levels, evaluating each sub-tree on V_{test} by traversing the tree for each sample, taking $O(|V_{test}|\cdot i)$ time at depth i, which yields $O(d \cdot |V_{test}|)$ complexity with early termination in practice; (2) *Bridge and branch extraction* (Lines 8-9) takes $O(|B|)$ time to extract bridge nodes and their child branches; (3) *Branch sorting* (Line 10) evaluates $|B|$ branches in $O(|B| \cdot |V_{test}| \cdot h_{avg})$ and sorts them in $O(|B| \log |B|)$; (4) *Normal sub-trees selection* (Lines 12-16) computes overall and per-class accuracy gains for each branch in $O(C \cdot |V_{test}| \cdot h_{avg})$, which consumes $O(|B| \cdot C \cdot |V_{test}| \cdot h_{avg})$ for all branches. Therefore, the overall time complexity of Algorithm 1 is $O\left(d \cdot |V_{test}| + |B| \cdot C \cdot |V_{test}| \cdot h_{avg} + |B| \log |B|\right)$, where the time complexity can be simplified to $O(|B| \cdot C \cdot |V_{test}|)$ for practical scenarios where n is the number of training samples.



(a) Ternary match input xbar. (b) TCAM resource overhead.

Figure 12: Switch resource consumption w/ and w/o optimization framework in Proteus.

7.5 Discussion

Scalability of Proteus. We analyze the scalability of Proteus's two-stage optimization framework. First, Proteus maintains low switch resource overheads as the topology size increases because the growth rate of critical paths is lower than that of the topology size. Second, Proteus consumes 0.13 s and 0.16 s for Internet2 and Fat-tree topologies in our experiments, respectively. The reason is that the two-stage decomposition of Proteus reduces problem complexity compared to joint optimization. In the future, we will develop heuristics for efficiently solving large-scale deployments. For example, we plan to develop both a Lagrange relaxation-based heuristic to minimize critical sub-tree copies and a greedy-based heuristic to minimize inter-sub-tree overheads.

Robustness of Proteus. Robustness against model drifts and adversarial manipulations is critical for real-world deployments. Proteus can achieve strong robustness through several mechanisms. First, the online tuning module continuously monitors classification accuracy, which enables timely detection of performance degradation caused by model drifts. Second, Proteus supports periodic model updates by recomputing sub-tree splitting and deployment decisions when accuracy drops below thresholds. Third, adversarial training can be applied before deployment to enhance model resilience against adversarial traffic manipulations.

Comparison with additive ML methods. Additive machine learning methods, such as XGBoost [6] and gradient boosting [12], improve classification accuracy by combining multiple weak learners during training. While these methods are effective for model training and interpretability, they address a different problem than Proteus. More precisely, these methods focus on how to train accurate models, while Proteus focuses on how to deploy a trained decision tree onto resource-constrained programmable switches. In this case, Proteus can deploy decision trees trained via a massive effective method, including XGBoost-derived trees, while additive ML methods cannot address the deployment challenges of limited switch resources, full flow coverage, and inter-sub-tree coordination that Proteus targets.

Support for ensemble models. Proteus can be extended to support multi-tree ensemble models. When users submit multiple decision trees, Proteus converts each tree into a P4 program comprising general match-action tables. These programs are then collectively compiled onto programmable switches, where each tree performs classification independently, and their results can be aggregated for final decisions. Thus, Proteus supports ensemble models by treating each tree as an independent deployment task.